

Autocorrect & Next-Word Prediction Keyboard

Single combined task: spelling correction (pyspellchecker) + context-aware next-word suggestion (n-grams) — explained line by line

This is **one keyboard system with two responsibilities**, matching both task descriptions given: **(1)** fix a misspelled word using the **spellchecker** library, and **(2)** suggest the most likely next word using the **preceding words as context**, via an **n-gram language model**. Every code block below is followed immediately by a line-by-line explanation. Install the one dependency first: `pip install pyspellchecker`.

How the two pieces fit together

When the user finishes a word, the keyboard **first** spell-checks and fixes that just-typed word (dictionary + edit-distance based), **then** feeds the corrected sentence into the n-gram model, which looks at the last 1–2 words to predict what's likely to come next. Correcting before predicting matters: if a typo were left in, the model wouldn't recognise the context and prediction would fail silently.

1. Imports and the training corpus

```
import re
from collections import defaultdict, Counter
from spellchecker import SpellChecker

CORPUS = """
I love to play cricket in the evening with my friends.
I love to play football in the morning with my brother.
...
Please remember to submit your assignment before the deadline.
"""
```

import re — Regex module — used to strip punctuation/non-letters during cleaning.

from collections import defaultdict, Counter — defaultdict auto-creates a Counter for any new context the first time it's seen; Counter counts word frequencies and gives us `.most_common()` for ranking — both are the backbone of the n-gram model.

from spellchecker import SpellChecker — The pre-trained English spelling-correction engine — no training needed, it ships with a built-in frequency dictionary.

CORPUS = """ ... """ — Sample sentences the n-gram model learns next-word patterns from (e.g. 'I love to' is always followed by 'play'). In production this would be a much larger real-world text dataset.

2. Cleaning and tokenizing text

```
def clean_text(text):
    text = text.lower()
    text = re.sub(r"^[a-z\s]", "", text)
    return text

def tokenize(text):
    return clean_text(text).split()
```

text = text.lower() — Case-folding — 'The' and 'the' must count as the same word.

text = re.sub(r"^[a-z\s]", "", text) — Regex substitution removing everything that isn't a lowercase letter or whitespace — strips punctuation/digits so 'friends.' becomes 'friends'.

return clean_text(text).split() — Cleans, then splits on whitespace into a list of word tokens — this feeds the n-gram counting step.

3. NgramModel — counting which word follows which context

```
class NgramModel:
    def __init__(self, n):
        self.n = n
        self.contexts = defaultdict(Counter)

    def train(self, tokens):
        for i in range(len(tokens) - self.n + 1):
            gram = tokens[i:i + self.n]
            context = tuple(gram[:-1])
            next_word = gram[-1]
            self.contexts[context][next_word] += 1
```

```
def predict(self, context, top_k=3):
    context = tuple(context[-(self.n - 1):])
    if context not in self.contexts:
        return []
    counter = self.contexts[context]
    total = sum(counter.values())
    ranked = counter.most_common(top_k)
    return [(word, round(count / total, 2)) for word, count in ranked]
```

self.contexts = defaultdict(Counter) — Frequency table: key = tuple of previous (n-1) words, value = Counter of which words followed and how often — this IS the n-gram model.

for i in range(len(tokens) - self.n + 1): — Slides a window of size n across all tokens (sliding-window technique), stopping before it runs past the list end.

context = tuple(gram[:-1]) / next_word = gram[-1] — Splits each n-word window into 'everything except the last word' (the context, must be a tuple to be a valid dict key) and 'the last word' (what we want to learn to predict).

self.contexts[context][next_word] += 1 — Counts one more occurrence of next_word following context — this is the Markov assumption in action: next word depends only on the last n-1 words.

if context not in self.contexts: return [] — If this exact context was never seen in training, admit 'no prediction' rather than guessing — the zero-frequency problem.

ranked = counter.most_common(top_k) / return [...] — Ranks candidate next-words by frequency and converts counts to probabilities (count/total) — giving $P(\text{next word} \mid \text{context})$.

4. NextWordPredictor — backing off from trigram to bigram

```
class NextWordPredictor:
    def __init__(self, corpus, max_n=3):
        self.vocab = set()
        self.models = {}
        tokens = tokenize(corpus)
        self.vocab.update(tokens)
        for n in range(2, max_n + 1):
            model = NgramModel(n)
            model.train(tokens)
            self.models[n] = model

    def predict_next(self, sentence, top_k=3):
        tokens = tokenize(sentence)
        for n in sorted(self.models.keys(), reverse=True):
            context_size = n - 1
            if len(tokens) >= context_size:
                context = tokens[-context_size:]
                suggestions = self.models[n].predict(context, top_k)
                if suggestions:
                    return suggestions
        return []
```

self.vocab = set() — Collects every unique word seen in training — reused later so the spellchecker also treats these domain words as 'known/correct'.

for n in range(2, max_n + 1): ... self.models[n] = model — Builds both a bigram (n=2) and a trigram (n=3) model from the same tokens, so the system can try the more specific model first and fall back if needed.

for n in sorted(self.models.keys(), reverse=True): — Tries trigram (3) before bigram (2) — this is **backoff**: prefer the model that uses more context, fall back to less context only if the specific one has never seen this phrase.

context = tokens[-context_size:] / suggestions = self.models[n].predict(...) — Takes the last context_size typed words and asks that n-gram model for predictions.

if suggestions: return suggestions — Stops at the first model size that actually has an answer.

5. SpellAutocorrector — fixing the just-typed word

```
class SpellAutocorrector:
    def __init__(self, extra_vocab=None):
        self.spell = SpellChecker()
        if extra_vocab:
            self.spell.word_frequency.load_words(extra_vocab)

    def correct_word(self, word):
        core = re.sub(r"^[A-Za-z]", "", word)
        if not core:
            return word
        if core.lower() not in self.spell.unknown([core]):
            return word
        fix = self.spell.correction(core)
        return word.replace(core, fix) if fix else word
```

self.spell = SpellChecker() — Loads the library's large pre-built English dictionary + word-frequency table.

self.spell.word_frequency.load_words(extra_vocab) — Teaches the spellchecker our corpus's own vocabulary (e.g. domain words), so it never wrongly 'corrects' a word the n-gram model already knows — this is how the two components share knowledge.

core = re.sub(r"^[A-Za-z]", "", word) — Strips punctuation stuck to the word (e.g. 'lern,' -> 'lern') so the dictionary lookup works, while the original 'word' is kept for reconstruction.

if core.lower() not in self.spell.unknown([core]): return word — self.spell.unknown() returns the subset of given words the dictionary does NOT recognise; if our word isn't in that set, it's already correctly spelled, so return it unchanged.

fix = self.spell.correction(core) — Returns the dictionary's best-guess correction — internally: generate candidates within edit-distance 1–2, pick the one with highest word frequency (Norvig's spelling-correction algorithm).

return word.replace(core, fix) if fix else word — Substitutes only the corrected letters back into the original chunk (preserving any trailing punctuation), or returns the word unchanged if no fix was found.

6. SmartKeyboard — combining both features

```
class SmartKeyboard:
    def __init__(self, corpus):
        self.predictor = NextWordPredictor(corpus)
        self.autocorrector = SpellAutocorrector(extra_vocab=self.predictor.vocab)

    def process(self, typed_text):
        words = typed_text.strip().split()
        if not words:
            return "", []
```

```

words[-1] = self.autocorrector.correct_word(words[-1])
corrected_text = " ".join(words)
suggestions = self.predictor.predict_next(corrected_text)
return corrected_text, suggestions

```

self.predictor = NextWordPredictor(corpus) — Trains the n-gram side once, at start-up.

self.autocorrector = SpellAutocorrector(extra_vocab=self.predictor.vocab) — Builds the spellchecker side, sharing the predictor's vocabulary so both parts of the system agree on what counts as a 'known' word — this line is literally where the two tasks are merged into one.

words[-1] = self.autocorrector.correct_word(words[-1]) — Step 1: fix only the most-recently-typed word (realistic — you correct what you just finished typing, not earlier accepted words).

corrected_text = " ".join(words) — Rebuilds the sentence with the correction applied.

suggestions = self.predictor.predict_next(corrected_text) — Step 2: now that the sentence is spelling-clean, ask the n-gram model what word is likely to come next — this ordering (correct, THEN predict) is why the two pieces work together correctly.

return corrected_text, suggestions — Returns both results together — what a real keyboard would show: the corrected text plus a next-word suggestion strip.

7. Demo and output

```

if __name__ == "__main__":
    keyboard = SmartKeyboard(CORPUS)
    test_inputs = [
        "I love to",
        "I want to lern",
        "Machin lerning is",
        "The weather is very",
    ]
    for typed in test_inputs:
        corrected_text, suggestions = keyboard.process(typed)
        print(f"You typed:      {typed}")
        print(f"Autocorrected:  {corrected_text}")
        print(f"Next word tips: {suggestions}")
        print("-" * 55)

```

test_inputs = [...] — Includes both a clean phrase and typo'd phrases ('lern', 'Machin lerning') to exercise both features in one run.

keyboard.process(typed) — Runs the full pipeline: spell-fix the last word, then predict the next word from context.

Actual output

```

You typed:      I love to
Autocorrected:  I love to
Next word tips: [('play', 1.0)]
-----
You typed:      I want to lern
Autocorrected:  I want to learn
Next word tips: [('how', 1.0)]
-----
You typed:      Machin lerning is
Autocorrected:  Machin lerning is
Next word tips: [('very', 0.5), ('a', 0.5)]

```

```
-----
You typed:      The weather is very
Autocorrected:  The weather is very
Next word tips: [('nice', 0.5), ('cold', 0.5)]
-----
```

'lern' -> 'learn' fixes the typo since it's the LAST word typed, and the model then correctly predicts 'how' next (from 'I want to learn how...' seen in training). 'Machin lerning is' isn't touched because the last word ('is') is already correct — only the most recently typed word gets autocorrected, exactly like a real keyboard. Probabilities like 0.5/0.5 reflect genuine 50/50 ambiguity in the small training corpus (e.g. 'the weather is very' was followed equally by 'nice' and 'cold').

8. Summary of concepts

Concept	Where used
Tokenization & case-folding	<code>clean_text()</code> , <code>tokenize()</code>
Markov assumption / sliding window	<code>NgramModel.train()</code>
Counting & probability from frequency	<code>NgramModel.predict()</code>
Backoff across model sizes	<code>NextWordPredictor.predict_next()</code>
Dictionary + edit-distance spell correction	<code>SpellAutocorrector</code>
Shared vocabulary between components	<code>SmartKeyboard.__init__()</code>
Correct-then-predict pipeline order	<code>SmartKeyboard.process()</code>